

```
fn show(&p: Point) { // has an immutable reference to a Point that
```

```

    someone else owns
    Println!("{}, {}, {}", p.x, p.y);
    // p does not get deallocated

Mutable reference &mut
fn main() {
    let p = Point{x: 1, y:2};
    scale(&mut p, 10); // main mutably-lends p to scale (=scale mutably-borrows p)
    fn scale(p: &mut Point, factor: u32) {
        p.x*=factor;
        p.y*=factor;
    }
}

Threads
use std::thread;
fn main() {
    // Thread body provided as a zero-argument closure

```

```
let t1 = thread::spawn(|| {
    // indicates that the closure takes no arguments
    println!("I am thread 1!"); });
// join() returns a Result, which can be Ok or Err
// unwrap() panics if the result is not Ok
t1.join().unwrap();
```

A Rust closure borrows data by default. But a thread cannot easily borrow data because the thread might outlive the thread that launched it. Instead, **move causes all accessed data to be moved**. But then it **can't be shared between multiple threads**, or by the launching

- **Arc** (Atomically reference-counted pointers):
 - An `Arc<T>` can be cloned
 - Each clone() calls hands out a new pointer to a piece of data, and increases a reference count
 - When one of these pointers goes out of scope, the reference count is decremented
 - When reference count reaches 0, the object gets **deallocated**
 - `Arc<T>` **manipulates reference count using atomics** => **thread safe**
 - `Arc<T>` allows **multiple** threads to access **read-only** shared data.
- Allowing unrestricted mutable access could lead to data races.
- For **shared mutable access between threads**: `Arc<MutableT>>`

- `Arc<T>` allows a mutex to be given to many threads
- `lock()` yields a mutex guard, which provides exclusive access to the protected data until the guard goes out of scope

Interior mutability:

`Arc<T>` does not allow mutable access to the referenced object. But `Arc<Mutex<T>>` allowed us to get a mutex guard and then modify the guarded data!

Achieved using interior mutability:

- `Mutex<T>` uses expert-written "unsafe" code to allow mutability, but only in a manner that avoids data races

Example:

```
let mut data = Vec::<u32>::new(); for _ in 0..max {data.push(1);}

// give multiple threads access to data (no modification allowed)
let data_reference_1 = Arc::new(data);
let data_reference_2 = data_reference_1.clone();
```

```
let mut result: Mutex<u32> = Mutex::new(0);
// Now Arc owns result, we should access result via Arc
let result_reference_1 = Arc::new(result);
let result_reference_2 = result_reference_1.clone();
let result_reference_3 = result_reference_1.clone();

let t1 = thread::spawn(move || {
    let mut my_result: u32 = 0;
```

```

// Mutex-protect the update to result
*result_reference_1.lock().unwrap() += my_result; });
// Same for t2

t1.join().unwrap(); t2.join().unwrap();
println!("{}", result_reference_3.lock().unwrap());
Atomic version:
let mut result: AtomicU32 = AtomicU32::new(0);

```

```
...
result_reference_1.fetch_add(my_result, Ordering::Relaxed);
// same for t2
...
Result_referece_3.load(Ordering::Relaxed)
```

Mutexes:
A mutex is associated with a certain object that it protects.

Mutex-protected hash set buckets

No way to directly access a bucket: you have to get its mutex

A mutex can only protect one object.

Dynamic Data Race Detection

Dynamic data race detection workflow

Each library function updates dynamic race detection state. This state allows the conditions for data races to be checked as the program runs. The library functions must be mutually exclusive. Otherwise we get

data races in the race detector!

Instrumentation leads to significant runtime overhead.

Vector clock-based race detection

A thread's **logical clock**: a non-negative integer. A thread's logical clock value **increases** each time the thread releases a mutex. Releasing a mutex indicates transitioning to next phase of concurrent program.

A **vector clock** is a tuple of N logical clocks, one per thread:
 $\langle c_0, c_1, \dots, c_{N-1} \rangle$. Equivalently: vector clock is a mapping from Threads to natural numbers. If V is a vector clock, $V(t)$ denotes logical clock of thread t . Set of all vector clocks: VC

